

Applications and Integration

Domain 2 of the CCDV-F blueprint — **33.1% of the exam, the single largest domain.**

Sources: Anthropic Partner Academy prep course (Module 1 — MSO Foundations, API-mechanics sections), [Using the Messages API](https://platform.claude.com/docs/en/build-with-claude/working-with-messages) (https://platform.claude.com/docs/en/build-with-claude/working-with-messages), [Vision](https://platform.claude.com/docs/en/build-with-claude/vision) (https://platform.claude.com/docs/en/build-with-claude/vision), [Claude on Google Cloud](https://platform.claude.com/docs/en/build-with-claude/claude-on-verte-x-ai) (https://platform.claude.com/docs/en/build-with-claude/claude-on-verte-x-ai), [Enterprise deployment overview](https://code.claude.com/docs/en/third-party-integrations) (https://code.claude.com/docs/en/third-party-integrations), [Create and distribute a plugin marketplace](https://code.claude.com/docs/en/plugin-marketplaces) (https://code.claude.com/docs/en/plugin-marketplaces), [Claude Code settings](https://code.claude.com/docs/en/settings) (https://code.claude.com/docs/en/settings), [Models overview](https://platform.claude.com/docs/en/about-claude/models/overview) (https://platform.claude.com/docs/en/about-claude/models/overview).

Understanding Requirements (3.4%)

A business problem ("help support agents answer faster") is **not yet a requirement** — requirements are derived from it, and conflating the two is a common design mistake:

- **Functional requirements:** what the system must *do*, stated with enough detail to check — e.g. "classify each ticket into one of four queues; draft a reply citing the relevant policy; never auto-send without human approval." A vague goal can't be designed against or verified; a specific one becomes a line in an eval and a criterion at review. These drive prompt design, tool selection, and output-constraint choices (see `4_model_selection_prompting_context.md`).
- **Infrastructure requirements:** the non-functional constraints the deployment must satisfy — mostly *not* stated in the business problem, but derived by asking what it implies: **latency** (how fast, measured where the user actually is), **scale** (how many requests, at what peak), **residency** (where must data be processed, under which regulation), **identity** (who acts, under what credentials, what must be auditable). These four most often decide the deployment platform, and are cheapest to capture before a platform gets chosen for other reasons (e.g. team familiarity — see the deployment-platform gotcha below).

Document requirements in a short record covering the functional behaviors, the infrastructure constraints, and which regulation each constraint traces back to — this is what lets you defend a platform/architecture choice as *following from the requirements* to a reviewer who wasn't in the room when you gathered them, rather than defending it as "what we're familiar with."

A solution architecture maps functional requirements onto infrastructure choices: a "summarize 10,000 documents overnight, cost-sensitive" functional requirement maps to the Batch API infrastructure choice; a "respond to a live chat user" functional requirement maps to streaming. Getting this mapping backwards — picking the infrastructure pattern before pinning down the actual requirement — is the root cause behind most of the "why is this slow/expensive" production complaints these systems generate.

Systems Life Cycle (2.8%)

A Claude application moves through the same lifecycle as any engineered system, with model-specific work mapped onto each phase:

Phase	What happens
1. Requirements	Capture functional and infrastructure needs (above)
2. Design	Choose the platform, the model, and the trust boundaries
3. Build	Write the agent, tools, and prompts
4. Test	Evals, unit, integration, end-to-end checks (<code>5_eval_debugging_security.md</code>)

Phase	What happens
5. Deploy	Pin the version, gate promotion on the eval
6. Operate	Instrument cost, latency, errors; enforce guardrails
7. Iterate	Feed production findings back into requirements

A **gate** is the decision to move from one phase to the next, and it's where a regulated engagement retains control: you don't move design → build until the chosen platform satisfies the residency requirement; you don't move deploy → full production until the new version clears the eval against its pinned baseline score. Refusing to skip a gate under deadline pressure is what keeps the application reviewable later — placing engineering work in the wrong phase (e.g. discovering a residency requirement at the deploy gate instead of the requirements phase) is expensive precisely because it's discovered late.

Beyond the lifecycle's own structure, one property is specific to Claude applications: because model behavior can shift on a version bump (see Model Selection and Tradeoffs in `4_model_selection_prompting_context.md`) and prompts drift in effectiveness as usage patterns evolve, the **operate** phase is never "done" the way it can be for traditional software with a fixed spec. Build eval suites as part of the build phase, not as an afterthought — they're what makes iterate → operate tractable once the system is live: a model or prompt change gets validated against the same eval suite rather than manual spot-checking, and that same eval suite is the artifact that gates the deploy phase.

Claude API Mechanics (6.8%)

Messages, tools, and the request/response cycle

The Messages API is the foundation everything else builds on: you send a `messages` array (alternating `user` / `assistant` turns) plus a `system` prompt and optional `tools` array; Claude returns content blocks — `text`, `tool_use`, or (on supporting models) `thinking`. A multi-turn tool-use exchange is you appending the `assistant` message containing `tool_use` blocks, then a `user` message containing matching `tool_result` blocks (matched by `tool_use_id`), and sending the whole growing array back each time — **the API is stateless per-request**; your application owns conversation state, not Anthropic's servers (unless you're using session-management features layered on top, like the Agent SDK's session store).

Vision

Image token cost — calculate before you commit, don't discover it in production. Claude views images in 28×28-pixel patches: cost in visual tokens is $\lceil \text{width}/28 \rceil \times \lceil \text{height}/28 \rceil$. A 1000×1000px image is 36×36 patches ≈ 1,296 tokens — ten high-resolution screenshots can consume as much context as a detailed system prompt. Each model tier has its own maximum native resolution (long-edge limit and visual-token limit); images beyond either are downscaled before the formula applies, and the limits have changed across model generations — confirm current per-tier numbers at build time. Measure the token cost of a *typical production image* against your context budget at design time — the fix for an over-budget image pipeline is often a ten-minute resize step if caught early, and considerably more expensive to retrofit post-deployment.

Images are supplied as content blocks with one of three source types:

- **base64**: image bytes inline, with `media_type` (`image/jpeg`, `image/png`, `image/gif`, `image/webp`) and `data` fields. No upload step, but the full payload re-sends **on every turn** — right for a one-off image where an upload step isn't worth the complexity; the same image sent repeatedly multiplies cost, so switch methods if reuse is likely.
- **url**: reference an externally-hosted image directly — no payload travels with the request, but you take on the dependency that the URL is stable, public, and reachable *at the moment Claude fetches it*. Skip for anything behind auth or short-lived signed URLs.

- `file` (via the Files API, currently beta, **not available on Bedrock or Vertex AI** — verify for your deployment platform): upload once, reference by `file_id` thereafter — overhead drops to near-zero after the first request. Right when the same asset appears across multiple requests/turns, or when asset management should live separately from inference calls.

PDFs use a `document` block, not `image` — same source pattern (`base64` / `url` / `file_id`), no required `name` field, optional `title` and `context` fields. All other mechanics (token cost, Files API reuse) apply identically to images.

```
{"type": "document", "source": {"type": "base64", "media_type": "application/pdf", "data": "<base64-bytes>"}, "title": "contract_review.pdf"}
```

Streaming with tool use — accumulate before you act

Streaming a plain text response is straightforward (reassemble text deltas as they arrive), but **streaming a response that includes tool calls needs extra handling**. In a non-streaming call, a `tool_use` block arrives complete and ready to read. In a streaming call, it arrives as a sequence of server-sent events, and the tool's input JSON accumulates across multiple `content_block_delta` events before it's complete — a `tool_use` block is not safe to act on until the stream closes and its full `input_json` has been reassembled. Acting on a partial block produces malformed tool inputs.

```
def stream_with_tools(client, **kwargs):
    tool_blocks = {}          # index -> accumulated block
    text_chunks = []
    with client.messages.stream(**kwargs) as stream:
        for event in stream:
            if event.type == "content_block_start":
                b = event.content_block
                tool_blocks[event.index] = {"type": b.type, "id": getattr(b, "id", None),
                                           "name": getattr(b, "name", None), "input_json": ""}
            elif event.type == "content_block_delta":
                d = event.delta
                if d.type == "input_json_delta":
                    tool_blocks[event.index]["input_json"] += d.partial_json
                elif d.type == "text_delta":
                    text_chunks.append(d.text)
            elif event.type == "message_stop":
                break
    # reconstruct completed tool calls only after the stream closes
    tool_calls = [{"id": b["id"], "name": b["name"], "input": json.loads(b["input_json"])}
                  for b in tool_blocks.values() if b["type"] == "tool_use"]
    return "".join(text_chunks), tool_calls
```

The same retrievable-vs-terminal failure handling from `5_eval_debugging_security.md` applies here: a stream that breaks mid-response is a **transient failure** — retry the whole request, never pass a partial accumulated block downstream as if it were complete.

Extended thinking and adaptive thinking

Two distinct reasoning mechanisms that do **not** both apply to the same model (see the per-model table in `4_model_selection_prompting_context.md`): **adaptive thinking** (Fable 5/Opus 5/Sonnet 5 — the model decides depth, tuned by `effort`) vs. **extended thinking** (`thinking.type: "enabled"`, currently Haiku 4.5 — produces explicit `thinking` content blocks). Thinking content is omitted from the response by default on the newest models; request summarized display explicitly when you need to show reasoning to a user or auditor.

Prompt caching

Covered in depth in `4_model_selection_prompting_context.md` (Cost and Token Management) — the API-mechanics-level summary: `cache_control` on a content block marks a prefix as cacheable, at a 5-minute or 1-hour TTL, up to 4 breakpoints per request. This is Claude API mechanics *and* a cost lever simultaneously — know both angles.

Deployment platforms: six places a Claude workload can run, and what actually decides between them

The customer's existing cloud usually decides the platform before technical merit does — this is a solution-architecture decision, not a procurement afterthought:

Platform	Identity/data model	Choose it when	Version pinning
First-party Claude API	Anthropic identity/terms	No binding cloud/residency constraint; want newest capabilities first (this platform typically gets new features earliest)	Pin the full model ID, retain the prior snapshot
Claude Platform on AWS	Anthropic identity/terms <i>via</i> the customer's AWS account; inference is Anthropic-operated, outside the AWS boundary	On AWS but want Anthropic's own model IDs/lifecycle and first-party feature parity	Same ID format as the first-party API
Claude in Amazon Bedrock	Messages API at <code>/anthropic/v1/messages</code> , broad feature parity (confirm feature-specific gaps against Bedrock docs); data stays inside the customer's AWS boundary	On AWS, wants feature parity, holds compliance posture there	Full model ID with <code>anthropic.</code> prefix; partner retirement dates differ from Anthropic's own schedule
Claude on Amazon Bedrock (legacy)	AWS identity/billing, <code>InvokeModel</code> / <code>Converse</code> APIs, ARN-versioned identifiers	Existing Bedrock integration not yet migrated to the Messages API	Pin via ARN-versioned model identifiers
Google Vertex AI	Google Cloud identity/IAM/billing; regional or global endpoints for residency	On Google Cloud, holds compliance posture there	Full model ID before rollout; partner retirement dates differ from Anthropic's
Third-party (e.g. Microsoft Foundry)	The wrapping product's own identity/billing	Customer already runs the platform embedding Claude	Per that platform's versioning controls

Microsoft Foundry gotcha: it offers Claude in **two hosting forms with different residency properties** — "Hosted on Azure" (currently Opus 4.8/Sonnet 5/Haiku 4.5, inference end-to-end on Azure infrastructure) vs. "Hosted on Anthropic" (all other Foundry Claude models, inference on Anthropic-operated infrastructure, and **not** sufficient for EU regional residency requirements). Residency must be confirmed **per model**, not per platform — the platform name alone doesn't tell you where a given model's inference actually runs. Same caution applies to Google Cloud's API shape specifics: model specified in the endpoint URL rather than the request body, `anthropic_version` a required body field (e.g. `vertex-2023-10-16`).

Version pinning: an alias is a moving target, a full model ID is a fixed snapshot

An alias (`opus`, `sonnet`) resolves to a recommended version that **updates over time and can differ by platform** — convenient, but an upstream update becomes a silent production change if nothing is watching for it. A pinned full model ID is fixed until you change the line yourself.

```
model = "claude-haiku-4-5"           # alias - can move without you knowing
model = "claude-haiku-4-5-20251001"  # pinned snapshot - fixed until you edit this line
```

For Claude 4.6 and later, the model ID alone pins to a specific snapshot (dateless format, still fixed); earlier models need the ID plus a date suffix. **Gate every version promotion on the eval suite:** send the new version to a slice of traffic, compare against the pinned baseline score, promote or roll back on the result — this is where the eval (Domain 4) stops being a one-time test and becomes the deployment gate. **Retain the prior pinned version** so a regression is a rollback, not a hotfix. Real incident pattern: a team shipped against a moving alias for convenience; the alias silently advanced, the response shape changed, a downstream parser threw `KeyError`, and there was no pinned prior version to roll back to — only a hotfix to the parser, leaving the unpinned deployment in place to fail the same way again.

Comparing platforms so the choice survives a procurement/security review

"Right for the customer's existing cloud" isn't yet an argument a security team signs off on — three dimensions, each requiring actual measurement, not assumption:

Dimension	How it differs	How to measure it correctly
Latency	An in-region cloud platform shortens round-trip time; the first-party API is typically advantaged on earliest feature access	From the customer's actual region , against their actual payload — a measurement from your own laptop hides the round-trip penalty that appears once the workload runs where the customer actually is
Compliance	Data residency, certifications, and audit controls are determined by the platform, not your code — for a regulated customer this is usually pass/fail, not a tradeoff	Against the customer's existing certification/residency requirement, during scoping — not discovered at the security review after the build is complete
Cost	Per-token rates are broadly aligned across platforms; total cost moves on egress, platform fees, and integration effort	Total cost per call per platform, including egress and integration — a lower token price can cost more overall

Gotcha (real incident pattern): a team picked the platform they'd shipped on before because the migration was familiar and the deadline was close. The build passed every functional test. At the customer's security review, the reviewer asked where data was processed — the chosen platform didn't satisfy the customer's residency requirement, and a less-familiar platform the team had available (via its regional deployment options) would have. The integration had to be rebuilt on the compliant platform. **Familiarity answers whether the team can build quickly; it says nothing about whether the customer is allowed to run the result.** Surface the compliance constraint during scoping — checking early costs a conversation, checking late costs an entire rebuild.

Realtime vs. batch — the core tradeoff

See `4_model_selection_prompting_context.md` (Technical Fundamentals) for the full synchronous/streaming/async/batch breakdown. The one-line version for this domain: **realtime** (sync or streaming) when a user or downstream process is waiting on the result now; **Message Batches API** when the workload is latency-tolerant and cost matters more than turnaround (up to 24h completion, lower per-token cost). This exact tradeoff is Sample Question 1 in the official exam guide.

Batch API mechanics: a single batch call accepts up to **100,000 requests or 256MB** (whichever limit hits first) — submit once, get a `batch_id` back, poll for completion, download results when done. **Results return in arbitrary order, not submission order** — set a `custom_id` on each request to match a result back to its input. Use it for offline pipelines, eval runs, and bulk jobs; skip it for anything a user is actively waiting on.

Gotcha (real incident pattern): a developer hit rate limits on a nightly job and "fixed" it by splitting the input list into smaller chunks — still hit the same rate limits three nights running. **Chunking a list and looping over the synchronous endpoint is not batching — it's serialization with extra steps.** The API still sees one request per item, back to back, regardless of how the list is sliced; the rate limit doesn't care about chunk

boundaries. The Message Batches API is a **different submission model**, not a smaller batch size — switching to an actual batch submission (not a loop) is what removes the rate-limit pressure and unlocks the lower per-token cost.

Software Engineering Foundations (7.4%)

Standard engineering practice applied to Claude-application specifics:

- **REST APIs / JSON:** the Messages API is a REST API returning JSON — every Claude application inherits ordinary REST concerns (idempotency, status-code handling per `5_eval_debugging_security.md`, request/response schema validation).
- **Asynchronous programming:** Python's `AsyncAnthropic` client vs. TypeScript's natively Promise-based client (see `4_model_selection_prompting_context.md`) — async buys concurrency (handle other work while a request is in flight), not lower per-request latency.
- **Version control:** applies to prompts and configuration, not just code — see Configuration Management below.
- **SDLC integration:** eval suites belong in CI, the same way unit tests do — a prompt or model-version change should fail a build the same way a broken test does, not surface as a silent quality regression in production.
- **Code review:** for Claude-application code, review coverage should explicitly include the *prompt* and *tool schema* changes, not just the surrounding application logic — a schema change is a breaking-change risk for every caller of that tool.
- **Refactoring, small- and large-scale:** small-scale — tightening a tool description, splitting an over-broad tool into two narrower ones (see Tool Implementation in `3_claude_code_tools_mcp.md`). Large-scale — migrating a workflow architecture to an agent architecture (or the reverse) as requirements shift from predictable to open-ended, per `1_agents_and_workflows.md`.
- **Packaging for reuse:** a working build and a *reusable* build are different finishing states. Packaging means separating engagement-/customer-specific values out into documented, parameterized configuration (with defaults) and bundling the eval suite alongside the code, so a future team configures the asset instead of reading the whole implementation to figure out what's safe to change. Real incident pattern: a team hardcoded customer-specific values (repo path, thresholds, prompt fragments) to hit a deadline, shipped a working agent template, and never revisited it — a second team picking it up months later found nothing to configure, no documentation of which values were customer-specific vs. load-bearing, and no bundled eval to confirm a guessed edit still worked, and ended up rewriting from scratch. **The cost of not parameterizing doesn't show up until someone else tries to reuse the build** — package while the build is fresh, since the knowledge of what's customer-specific is cheapest to write down before the people who have it move on to the next engagement.

Claude Application Design (8.6%)

How Claude interprets instructions across interfaces

The same model, different instruction hierarchies depending on the interface:

Interface	Persistent instructions come from	Notes
Claude Code	<code>CLAUDE.md</code> hierarchy (see <code>3_claude_code_tools_mcp.md</code>)	Re-injected every request; cached
Claude Desktop / claude.ai	Custom instructions / Projects	User- or workspace-scoped, not file-based

Interface	Persistent instructions come from	Notes
API / SDKs	<code>system</code> parameter you set per-request	Fully under your application's control; no implicit persistence unless you build it

A design mistake specific to this domain: assuming behavior tuned/tested against one interface (e.g., a CLAUDE.md-driven Claude Code workflow) transfers unchanged to the raw API, where there's no CLAUDE.md at all and every persistent instruction must be explicitly re-sent as part of `system` on every call.

Content boundaries

The same isolation discipline from indirect-prompt-injection defense (`5_eval_debugging_security.md`) is fundamentally an **application design** concern, not just a security one: deciding what content Claude is allowed to treat as instruction-bearing (your system prompt, your own user's direct input) versus what it must treat as inert data (retrieved documents, tool results, third-party content) is a design decision made when you architect the message flow, not a patch applied after an incident.

Schema design

Tool input/output schemas and structured-output JSON schemas (see Output Handling in `4_model_selection_prompting_context.md`) are part of your application's API contract — treat schema changes with the same discipline as a public API's breaking-change policy, because a schema change can silently break every caller relying on the old shape, with no compiler to catch it.

Session hygiene

For multi-turn applications: decide explicitly how long a session/conversation lives, when to summarize or fork it (see Session management in `1_agents_and_workflows.md`), and how stale sessions get cleaned up. An unbounded session that never resets accumulates context (cost, latency) and drifts (see context engineering, `4_model_selection_prompting_context.md`) — session hygiene is the operational discipline that keeps both in check.

Plugin management

A Claude Code **plugin** bundles together the components from `3_claude_code_tools_mcp.md` (commands, agents, skills, hooks) as one distributable, installable unit, described by a `.claude-plugin/plugin.json` manifest. A **marketplace** is a repository exposing a `.claude-plugin/marketplace.json` listing multiple plugins with their sources; installing from a marketplace can **auto-install a plugin's declared dependencies** (dependency resolution declared in `plugin.json` , with marketplace-entry fields able to override/supplement it). Plugin management as an application-design concern means deciding what's bundled as a reusable plugin (shared across projects/teams) versus what's project-local configuration (CLAUDE.md, `.claude/settings.json` in one repo) — see Configuration Management below for that boundary.

Configuration Management (4.1%)

Configuration management for a Claude application spans four distinct artifacts, each with its own versioning discipline:

- **CLAUDE.md**: project/user conventions and persistent context — version-controlled like code (see the hierarchy table in `3_claude_code_tools_mcp.md`).
- **settings.json**: tool permissions, hooks, MCP server registration — same five-scope precedence as `3_claude_code_tools_mcp.md` covers (managed > CLI > local > project > user), with the **permissions accumulate, don't override** exception.

- **Model version pinning:** every Claude model ID is a pinned snapshot (dated, e.g. `claude-haiku-4-5-20251001`, or — starting with the 4.6 generation — a dateless-but-still-pinned format). Pinning protects production from an unannounced behavior shift on model upgrade; it also means **you** own the decision of when to move to a newer pin, verified against your eval suite rather than assumed safe.
- **Prompt versioning:** system prompts and few-shot examples are production configuration, not throwaway text — track them the same way as code (version control, changelogs, rollback capability), because a "small wording tweak" can measurably shift output distribution (see non-determinism/evals, `4_model_selection_prompting_context.md`).
- **Plugin dependencies:** declared in `plugin.json`, resolved (and optionally auto-installed) at plugin-install time via the marketplace listing — a plugin's own dependency graph is configuration you inherit, not just the plugin's headline capability.

The common thread across all four: **treat prompt/config changes with the same rigor as code changes** — version control, review, and eval-suite validation before a change reaches production — because none of these artifacts are compiled or type-checked, so nothing else will catch a regression before your users do.